

FORM: SUM-TL-01

Taller – Modelado de Comunicación entre Procesos Distribuidos

Materia: Aplicaciones Distribuidas

Carrera: Software

Docente: Ing. Guerrero Ulloa Gleiston Cicerón

Fecha: 19 de junio de 2026

Integrantes:

Bedón Viteri Keyla Betzabé
Castro López Pedro Leonardo
Emanuel Pino Juliana Romina
Viñueza Sánchez Harold Nicolás

1 Contexto y escenario de negocio adoptado

El taller FORM: SUM-TL-01 tiene carácter formativo y antecede a la Práctica Experimental de la Unidad 1 (PE-U1). Su propósito es que el equipo modele, mediante un diagrama de comunicación entre procesos, el tipo de comunicación, el protocolo y el middleware utilizados en un escenario de aplicaciones distribuidas, justificando la elección realizada.

1.1 Escenario adoptado: “UTEQ Connect” – red social de mensajería interna

Tal como se identificó en la retroalimentación de la primera entrega, el equipo formaliza como escenario de negocio una **red social de mensajería interna para estudiantes**, a la que llamaremos **UTEQ Connect**. El núcleo visible de este escenario es exactamente el chat en tiempo real ya implementado en la Práctica 2 (WebSocket + STOMP), que permite a dos o más usuarios conversar de forma instantánea. El resto de mecanismos de comunicación que el equipo ya implementó y verificó durante la Unidad 1 no se descartan ni se presentan de forma aislada: se reinterpretan como los **servicios de backend** que sostendrían a UTEQ Connect en producción, tal como se resume en la Tabla 1.

Tabla 1: Mapeo de cada paradigma ya implementado a un componente del escenario UTEQ Connect

Paradigma	Componente de UTEQ Connect	Rol dentro del escenario
Mensajería (Web-Socket+STOMP)	Chat en tiempo real (cara al usuario)	Núcleo funcional: envío y recepción instantánea de mensajes entre usuarios conectados.
Sockets TC-P/UDP	Servicio de presencia y <i>heartbeat</i>	Verifica qué usuarios están en línea (TCP) y envía pings de baja prioridad (UDP) entre microservicios internos.
RPC – Thrift	Servicio interno de métricas	Calcula estadísticas de uso (mensajes enviados, tiempo de actividad) compartidas entre módulos backend escritos en distintos lenguajes.
gRPC	Servicio de notificaciones push	Envía alertas en <i>streaming</i> (ej. “tienes 3 mensajes nuevos”) y soporta canales bidireccionales para notificaciones interactivas.
RMI	Módulo administrativo (backoffice)	Operaciones de soporte sobre cuentas de usuario, ejecutadas por personal administrativo dentro del mismo ecosistema Java.

Esta reinterpretación no altera el código ya implementado y verificado (Tabla 2): únicamente formaliza, con un nombre y un dominio concretos, el propósito de negocio que cada mecanismo cumpliría dentro de una misma plataforma, en lugar de presentarlos como cinco ejercicios desconectados entre sí.

Los cinco paradigmas documentados, ya situados dentro de UTEQ Connect, son:

1. **Sockets** – TCP y UDP (*practica1-sockets*) – presencia/heartbeat.
2. **RPC clásico** – Apache Thrift (*Guia03-RPC*) – métricas internas.
3. **RPC moderno con streaming** – gRPC (*Guia04-gRPC*) – notificaciones push.
4. **RMI** – Java Remote Method Invocation (*Guia05-RMI*) – backoffice administrativo.
5. **Mensajería** – WebSocket + STOMP (*demo*, chat en tiempo real) – núcleo del producto.

2 Marco teórico

2.1 Comunicación entre procesos distribuidos

En un sistema distribuido, dos o más procesos que se ejecutan en máquinas distintas deben intercambiar información como si formaran parte de un mismo programa. Esto exige acordar, de antemano, un protocolo (las reglas de bajo nivel del intercambio de bytes) y, opcionalmente, un middleware que oculte la complejidad de la red al programador [5]. Además, como los procesos no comparten un reloj físico común, ordenar correctamente los eventos que ocurren en distintas máquinas es un problema clásico de los sistemas distribuidos, formalizado por Lamport mediante relojes lógicos [11].

2.2 Sockets

Un *socket* es el punto final de una comunicación de red identificado por una dirección IP y un puerto, sobre la arquitectura de redes de conmutación de paquetes propuesta originalmente por Cerf y Kahn [9]. TCP (RFC 793) ofrece un canal orientado a conexión, confiable y con orden garantizado; UDP (RFC 768) ofrece un canal sin conexión, más rápido pero sin garantías de entrega [1, 2]. No existe middleware: la aplicación debe definir su propio protocolo sobre los bytes crudos.

2.3 RPC (Remote Procedure Call)

RPC permite invocar una función que se ejecuta en un proceso remoto como si fuera una llamada local, idea formalizada por Birrell y Nelson en su implementación clásica de RPC [10]. Requiere un contrato (*Interface Definition Language*, IDL) compartido entre cliente y servidor, a partir del cual un *middleware* genera automáticamente el código de *marshalling/unmarshalling* (Apache Thrift [6]) o utiliza serialización binaria sobre HTTP/2 (gRPC, sobre RFC 7540 [4]), este último ampliamente adoptado en arquitecturas de microservicios modernas [12].

2.4 RMI (Remote Method Invocation)

RMI extiende la idea de RPC al paradigma de objetos: un cliente Java invoca métodos de un objeto que reside en otra JVM, a través de un *stub* obtenido de un servicio de nombres (RMI Registry), sobre el protocolo JRMP [5].

2.5 Mensajería y middleware orientado a mensajes

La mensajería desacopla emisor y receptor mediante un intermediario (*message broker*) que gestiona suscripciones a destinos y reenvía cada mensaje publicado a todos los suscriptores interesados, sin que el emisor conozca su identidad. WebSocket (RFC 6455 [3]) provee el transporte full-dúplex y STOMP [7] el protocolo de mensajería que viaja sobre él.

3 Resumen de la arquitectura distribuida implementada

La Tabla 2 sintetiza la evidencia técnica que sustenta cada diagrama de la Sección 4: el módulo del repositorio, el puerto utilizado y el comportamiento observado durante la ejecución real documentada en las guías de práctica.

Tabla 2: Evidencia de ejecución por paradigma de comunicación

Paradigma	Módulo	Puerto	Comportamiento verificado
Sockets TCP	ServidorTCP / ClienteTCP	9000	Servidor multihilo (<code>ExecutorService</code>); comandos <code>HORA</code> , <code>ECO</code> , <code>SALIR</code> respondidos correctamente.
Sockets UDP	ServidorUDP / ClienteUDP	9001	ACK con timestamp por datagrama; timeout de 3000 ms en el cliente ante pérdida de respuesta.
RPC (Thrift)	ServidorRPC / ClienteRPC	9090	Operaciones <code>sumar</code> , <code>restar</code> , <code>multiplicar</code> , <code>dividir</code> ejecutadas remotamente sobre <code>calculadora.thrift</code> .
gRPC	ServidorGRPC / ClienteGRPC	50051	Tres modos probados: llamada unaria, <i>server streaming</i> (3 noticias) y chat bidireccional.
RMI	ServidorRMI / ClienteRMI	1099	<code>Naming.lookup()</code> obtiene el stub; <code>RemoteException</code> propagada correctamente en división por cero.
Mensajería	WebSocketConfig / ChatController	8080	Dos clientes web independientes intercambian mensajes en tiempo real vía <code>/topic</code> y <code>/app</code> .

4 Modelado de la comunicación entre procesos

4.1 Cuadro de clasificación por aspecto

Antes de entrar al detalle de cada mecanismo, la Tabla 3 y la Tabla 4 clasifican los cinco paradigmas ya implementados aspecto por aspecto (tipo de comunicación, protocolo base, serialización, middleware, modo de operación, sincronización causal [11] y puertos reales utilizados), en el mismo formato denso usado para comparar mecanismos de comunicación en la literatura técnica.

Tabla 3: Clasificación por aspecto – mecanismos de bajo nivel y mensajería

Aspecto	Sockets (TCP/UDP)	Mensajería (WebSocket+STOMP)
Tipo de comunicación	Sockets (nivel transporte)	Mensajería (publicar/suscribir)
Protocolo base	TCP/IP y UDP/IP directos	WebSocket (full-dúplex) + STOMP
Serialización	Texto plano (comandos propios)	Texto plano (tramas STOMP)
Middleware	Ninguno (<i>raw sockets</i>)	Spring SimpleBroker
Modo de operación	Síncrono, con ACK manual (TCP) o sin ACK (UDP)	Asíncrono, <i>publish/subscribe</i>
Sincronización causal	Ninguna (timestamp de reloj físico del sistema, sin orden causal entre procesos)	Ninguna (orden de llegada al <i>broker</i> , sin reloj lógico)
Puertos utilizados	9000 (TCP), 9001 (UDP)	8080

Tabla 4: Clasificación por aspecto – mecanismos basados en contrato (RPC y RMI)

Aspecto	RPC – Thrift	RPC – gRPC	RMI
Tipo de comunicación	RPC (contrato IDL)	RPC moderno (contrato .proto)	Invocación remota de objetos
Protocolo base	TCP directo (TSocket)	HTTP/2	JRMP sobre TCP
Serialización	TBinaryProtocol (binario)	Protocol Buffers v3 (binario)	Serialización nativa de Java
Middleware	Apache Thrift (codegen desde .thrift)	gRPC Framework + Protobuf (codegen desde .proto)	RMI Registry + <i>stubs</i> dinámicos
Modo de operación	Síncrono (petición/-respuesta)	Síncrono unario + <i>streaming</i> (servidor/-bidireccional)	Síncrono (invocación de método remoto)
Sincronización causal	Ninguna (sin mecanismo de reloj lógico)	Reloj de Lamport real: ts = max(local, recibido) + 1, implementado en MensajeriaServiceImpl (llamadas unaria y bidireccional)	Ninguna (sin mecanismo de reloj lógico)
Puertos utilizados	9090	50051	1099

4.2 Sockets (TCP y UDP)

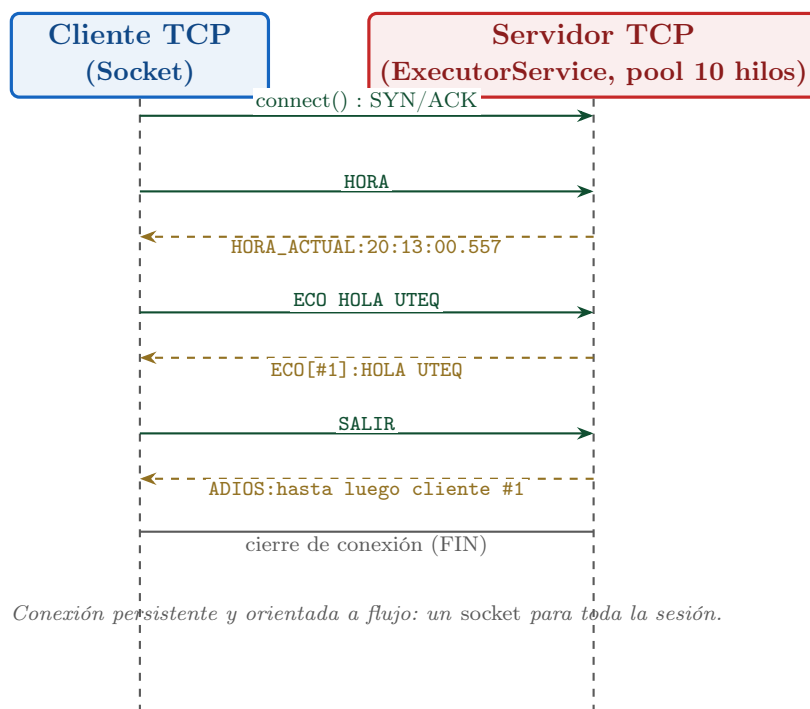


Diagrama 1a. Comunicación por Sockets TCP – modelo cliente/servidor orientado a conexión.

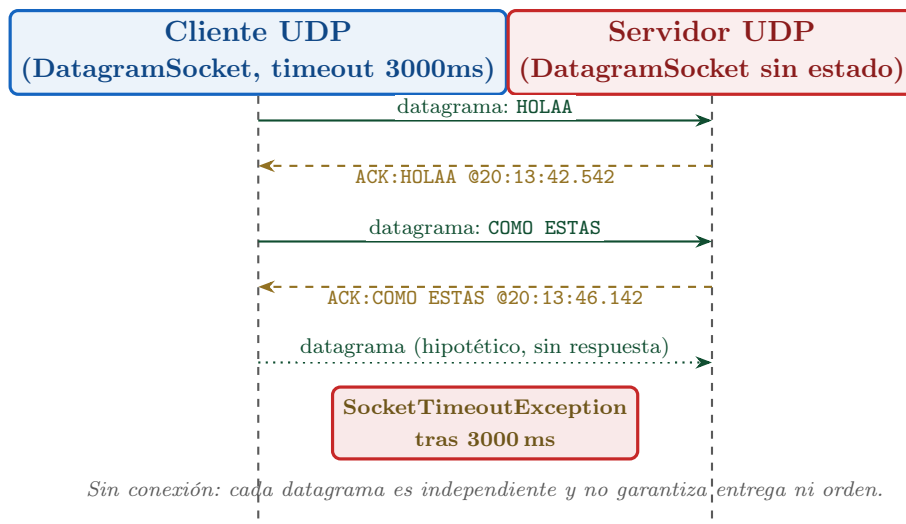


Diagrama 1b. Comunicación por Sockets UDP – modelo sin conexión, basado en datagramas.

Tipo de comunicación

Sockets de bajo nivel, síncronos punto a punto.

Protocolo TCP (RFC 793) para el canal orientado a conexión; UDP (RFC 768) para el canal sin conexión [1, 2].

Middleware

Ninguno. La comunicación ocurre directamente entre los *sockets* del sistema operativo, sin intermediarios ni capas de serialización adicionales.

Justificación para UTEQ Connect

El servicio de **presencia/heartbeat** (Tabla 1) necesita confirmar miles de veces por minuto si un usuario sigue conectado, con el menor costo posible por verificación. Sockets es la opción correcta porque permite definir un mini-protocolo a la medida (HORA/ECO/SALIR) sin el costo de compilar un IDL ni de mantener un canal HTTP/2.

Alternativa descartada

Se evaluó usar RPC (Thrift o gRPC) también para la presencia, pero se descartó: el costo de *marshalling* y de mantener un contrato formal es injustificable para un mensaje de un solo byte (¿estás ahí? sí/no). TCP se usa cuando importa confirmar la verificación (p. ej. cierre de sesión); UDP cuando basta un sondeo best-effort de baja prioridad.

4.3 RPC clásico – Apache Thrift

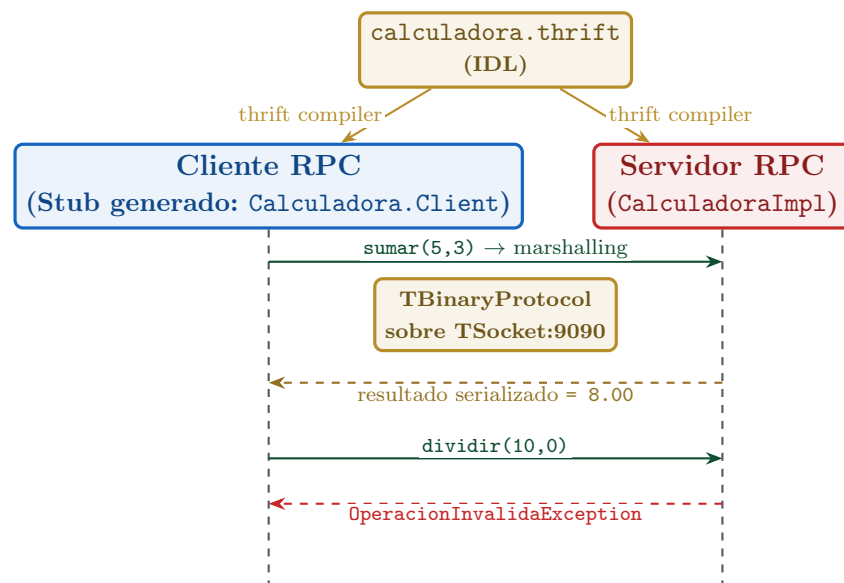


Diagrama 2. Comunicación por RPC clásico con Apache Thrift – generación de stub/skeleton a partir de un IDL.

Tipo de comunicación

RPC (*Remote Procedure Call*) síncrono, basado en contrato (IDL).

Protocolo TBinaryProtocol de Thrift sobre transporte TCP (TSocket).

Middleware

Apache Thrift: compilador IDL que genera automáticamente *stub* (cliente) y *skeleton/processor* (servidor) a partir de `calculadora.thrift`, junto con su *runtime* de serialización y transporte.

Listing 1: Contrato real calculadora.thrift del repositorio Guia03-RPC

```
1 namespace java ec.edu.uteq.distribuidas.rpc.gen
2
3 exception OperacionInvalidaException {
4     1: required string mensaje
5 }
6
7 service Calculadora {
8     double sumar      (1: double a, 2: double b)
9     double restar      (1: double a, 2: double b)
10    double multiplicar (1: double a, 2: double b)
11    double dividir      (1: double a, 2: double b)
12        throws (1: OperacionInvalidaException ex)
13    string nombreServidor()
14 }
```

Justificación para UTEQ Connect

El **servicio interno de métricas** (Tabla 1) es consumido por módulos backend que podrían estar escritos en distintos lenguajes (analytics en Python, panel admin en Java). Thrift resulta adecuado porque obliga a compartir exactamente el mismo contrato de operaciones (firmas, tipos, excepciones como `OperacionInvalidaException`), evitando que los módulos se desincronicen manualmente.

Alternativa descartada

Se consideró RMI para este servicio, pero se descartó por ser exclusivo de Java: si mañana el equipo de analytics de UTEQ Connect prefiere Python, RMI lo bloquearía por completo, mientras que Thrift solo exige regenerar el código cliente desde el mismo `.thrift`. El costo asumido es la dependencia de un paso de compilación previo del IDL.

4.4 RPC moderno con streaming – gRPC

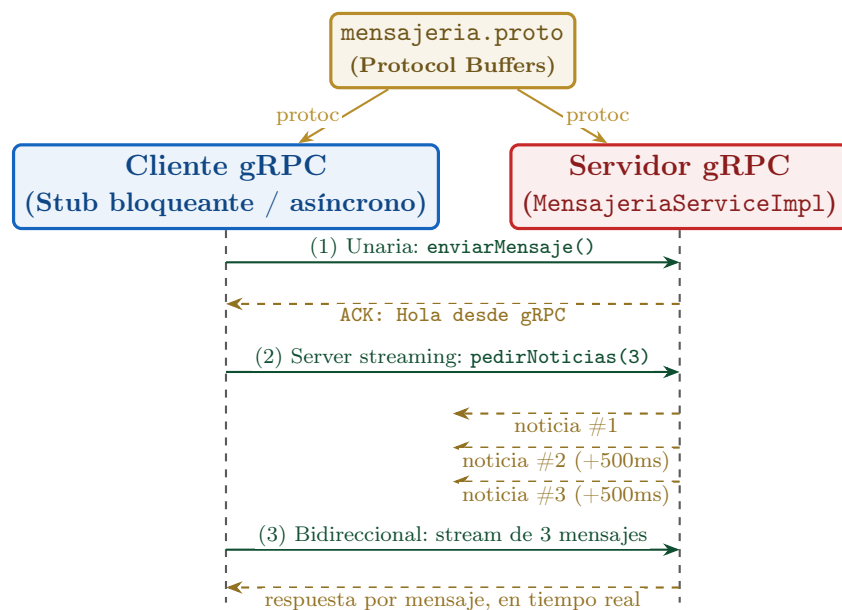


Diagrama 3. Comunicación por gRPC – llamada unaria, *server streaming* y chat bidireccional sobre HTTP/2.

Tipo de comunicación

RPC moderno con soporte nativo de *streaming* (unario, servidor, cliente y bidireccional).

Protocolo Protocol Buffers serializados sobre HTTP/2 (RFC 7540) [4].

Middleware

Framework gRPC + **protoc** (generador de código a partir de `mensajería.proto`), que provee canales multiplexados y manejo de *streams* concurrentes.

Listing 2: Contrato real `mensajería.proto` del repositorio Guia04-gRPC

```
1 syntax = "proto3";
2 package ec.edu.uteq.distribuidas.grpc;
3
4 message SolicitudMensaje {
5     string remitente = 1;
6     string contenido = 2;
7     int64 timestamp = 3;
8 }
9
10 message RespuestaMensaje {
11     string servidor = 1;
12     string contenido = 2;
13     int64 timestamp = 3;
14     bool exito = 4;
15 }
16
17 message SolicitudStream {
```

```

18  string usuario = 1;
19  int32 cantidad = 2;
20  }
21
22  service ServicioMensajeria {
23    rpc EnviarMensaje (SolicitudMensaje) returns (RespuestaMensaje);
24    rpc RecibirNoticias (SolicitudStream) returns (stream
        RespuestaMensaje);
25    rpc ChatBidireccional (stream SolicitudMensaje) returns (stream
        RespuestaMensaje);
26  }

```

Justificación para UTEQ Connect

El **servicio de notificaciones push** (Tabla 1) necesita enviar alertas incrementales (“tienes 3 mensajes nuevos”) y soportar canales interactivos sin reabrir conexión por cada evento. gRPC se eligió porque el chat bidireccional y el envío incremental de noticias (*server streaming*) no son naturales en Thrift clásico ni en sockets crudos, pero sí en gRPC gracias a HTTP/2.

Alternativa descartada

Se consideró reutilizar Thrift (ya usado para métricas) también para notificaciones, pero Thrift clásico no soporta *streaming* de servidor de forma nativa; habría exigido polling manual desde el cliente. gRPC es la opción más cercana a un estándar de microservicios moderno [12], por lo que ya se reutilizó en GA-SUM-03.

4.5 RMI – Java Remote Method Invocation

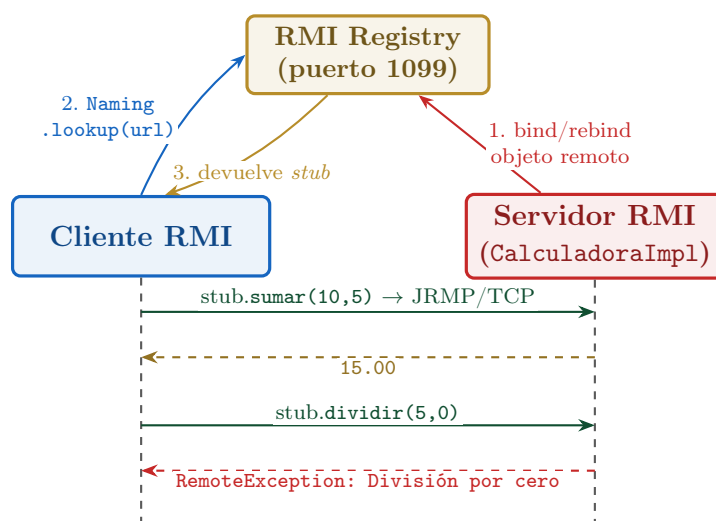


Diagrama 4. Comunicación por RMI – registro, búsqueda de *stub* e invocación remota transparente.

Tipo de comunicación

RMI (invocación remota de objetos, orientado a objetos distribuidos).

Protocolo JRMP (*Java Remote Method Protocol*) sobre TCP [5].

Middleware

`rmiregistry` (RMI Registry) como servicio de nombres, y los *stubs/skeletons* generados dinámicamente por la JVM para hacer transparente la llamada remota.

Justificación para UTEQ Connect

El **módulo administrativo (backoffice)** (Tabla 1) es de uso interno, exclusivo del equipo de soporte, y está escrito íntegramente en Java. RMI es la opción más natural porque permite invocar métodos de un objeto remoto *como si fuera local*, sin escribir código explícito de serialización, lo que acelera el desarrollo de pantallas administrativas internas.

Alternativa descartada

Se descartó gRPC para este módulo: introducir `.proto`, `protoc` y un canal HTTP/2 sería sobre-ingeniería para un backoffice cerrado, de bajo volumen y 100 % Java, donde la transparencia de RMI ya resuelve el problema con menos piezas móviles.

4.6 Mensajería – WebSocket + STOMP

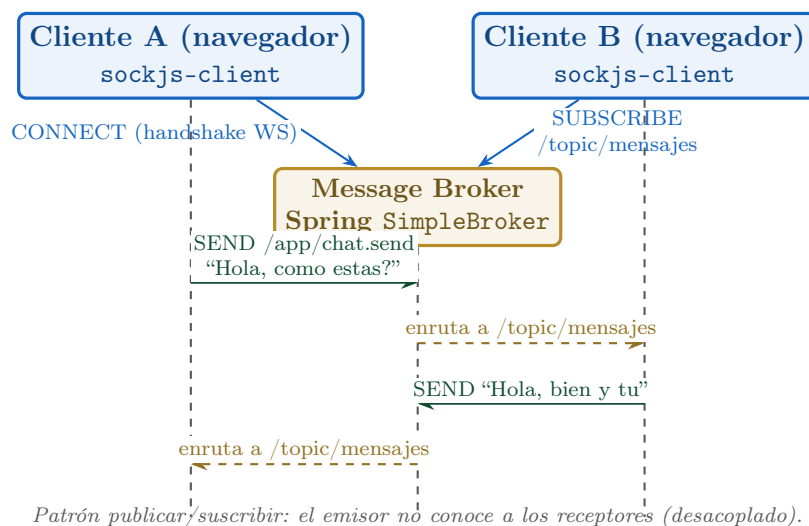


Diagrama 5. Comunicación por mensajería con WebSocket + STOMP – patrón publicar/suscribir mediado por un *broker*.

Tipo de comunicación

Mensajería asíncrona, patrón publicar/suscribir (*publish/subscribe*).

Protocolo WebSocket (RFC 6455) como transporte full-dúplex, con STOMP como protocolo de mensajería sobre él [3, 7].

Middleware

Message broker embebido de Spring (`SimpleBroker`), que gestiona las suscripciones a destinos (`/topic/mensajes`) y enruta cada mensaje publicado en `/app/chat.send` a todos los suscriptores.

Justificación para UTEQ Connect

El **chat en tiempo real** (Tabla 1) es el núcleo funcional de la red social: el número de receptores de un mensaje es variable y desconocido por el emisor (broadcast a todos los usuarios conectados a una sala), algo que RPC o RMI no resuelven de forma natural porque exigen conocer al destinatario de antemano.

Alternativa descartada

Se descartó modelar el chat con sockets crudos (como en la presencia): habría obligado al equipo a reimplementar manualmente la gestión de suscripciones y el *fan-out* de mensajes a N usuarios, exactamente lo que un *message broker* ya resuelve de fábrica.

5 Tabla comparativa de paradigmas

Tabla 5: Comparativa de tipo de comunicación, protocolo, middleware y caso de uso ideal

Paradigma	Acoplamiento	Protocolo	Middleware	Caso de uso ideal
Sockets (TCP/UDP)	Alto (cliente conoce IP:puerto exacto)	TCP / UDP	Ninguno	Protocolos propios de bajo nivel, control total
RPC – Thrift	Alto (contrato IDL compartido)	TBinaryProtocol / TCP	Apache Thrift	Microservicios multilenguaje con contrato fijo
RPC – gRPC	Alto (contrato .proto)	Protobuf / HTTP-2	gRPC + protoc	Microservicios modernos, streaming bidireccional
RMI	Alto (objetos remotos Java)	JRMP / TCP	RMI Registry	Ecosistemas 100 % Java, objetos distribuidos
Mensajería	Bajo (pub/sub, anónimo)	STOMP / Web-Socket	Message Broker	Notificaciones, chat, eventos uno-a-muchos

6 Selección y justificación para la Práctica Experimental U1

Para la PE-U1, el equipo recomienda continuar con **Sockets TCP/UDP** (Sección 4.2), es decir, el componente de **presencia/heartbeat** de UTEQ Connect, como base de la práctica experimental, por tres razones concretas:

1. **Evidencia de funcionamiento.** Ya existe una implementación funcionando y verificada (**ServidorTCP**, **ClienteTCP**, **ServidorUDP**, **ClienteUDP**), lo cual reduce el riesgo técnico de la siguiente entrega.
2. **Cobertura pedagógica completa.** Sockets obliga al equipo a diseñar su propio protocolo de aplicación (comandos **HORA/ECO/SALIR**), a diferencia de RPC, gRPC o RMI, donde gran parte del protocolo lo define el framework.
3. **Comparación TCP vs. UDP en un mismo entregable.** Permite mostrar en una sola práctica la diferencia entre comunicación orientada a conexión y sin conexión, requerimiento explícito de la rúbrica de PE-U1.

No se descarta el uso de RPC, gRPC, RMI o mensajería en prácticas experimentales posteriores (de hecho, gRPC ya se reutilizó en GA-SUM-03 para el diseño de microservicios), pero para la PE-U1 específicamente, Sockets es la opción más defendible frente al docente.

7 Notas para la presentación (5 minutos)

1. **(0:00–1:00) Contexto y escenario.** Presentar UTEQ Connect (Tabla 1) y explicar cómo cada paradigma ya implementado encaja en un componente real de esa red social.
2. **(1:00–3:00) Recorrido por los diagramas.** Mostrar brevemente los Diagramas 1 a 5, deteniéndose en la diferencia clave de cada uno: TCP/UDP (sin middleware), Thrift/gRPC (contrato IDL/proto), RMI (registro de nombres) y mensajería (broker pub/sub).
3. **(3:00–4:00) Tabla comparativa.** Presentar la Tabla 5 como síntesis visual rápida, incluyendo las alternativas descartadas por componente.
4. **(4:00–5:00) Recomendación para PE-U1.** Cerrar con la justificación de la Sección 6: Sockets (presencia/heartbeat) como base, con evidencia ya funcionando.

Pregunta crítica anticipada

Pregunta esperada de otro equipo: *“Si ya tienen gRPC funcionando, con streaming y mejor rendimiento que sockets crudos, ¿por qué no usar gRPC directamente para la PE-U1?”*

Respuesta preparada: gRPC introduce una capa adicional (definición `.proto`, compilación con `protoc`, dependencia de HTTP/2) que oculta justamente lo que la PE-U1 evalúa: el manejo explícito de `sockets`, puertos y framing de mensajes a bajo nivel. Sockets es la opción pedagógicamente correcta para esta práctica puntual; gRPC es la opción de ingeniería correcta para un sistema productivo, y por eso ya se usó en GA-SUM-03.

8 Estructura del proyecto

El Listado 3 muestra el árbol real (simplificado) del repositorio **Aplicaciones-Distribuidas**, con un módulo Maven/Gradle independiente por cada paradigma documentado en este taller. Nótese que `practical1-sockets` aparece como una carpeta anidada dentro de sí misma (`practical1-sockets/practical1-sockets/`): esto ocurre porque el proyecto se generó originalmente a partir de un arquetipo Maven (`archetype-resources`), que crea automáticamente un subdirectorio con el nombre del artefacto dentro de la carpeta del repositorio. Se conserva la estructura real, sin corregirla artificialmente, como evidencia fiel del repositorio `practical1-sockets`.

Listing 3: Árbol de archivos real del repositorio (simplificado)

```
1 Aplicaciones-Distribuidas/  
2   practical1-sockets/practical1-sockets/    # Sockets TCP/UDP (  
3     subcarpeta anidada)  
4   .idea/  
5   src/main/  
6     java/ec/edu/uteq/distribuidas/  
7       tcp/  
8         ClienteTCP.java  
9         ServidorTCP.java  
10      udp/  
11        ClienteUDP.java
```

```

11      ServidorUDP.java
12      resources/
13      META-INF/maven/
14      archetype-resources/
15      .gitignore
16      pom.xml
17  Guia03-RPC/                                # RPC clasico (Thrift)
18      src/main/thrift/calculadora.thrift
19      src/main/java/.../rpc/server/ServidorRPC.java
20      src/main/java/.../rpc/client/ClienteRPC.java
21  Guia04-gRPC/                                # RPC moderno (gRPC)
22      src/main/proto/mensajeria.proto
23      src/main/java/.../grpc/server/ServidorGRPC.java
24      src/main/java/.../grpc/client/ClienteGRPC.java
25  Guia05-RMI/                                # RMI
26      src/main/java/.../rmi/api/CalculadoraRemota.java
27      src/main/java/.../rmi/server/ServidorRMI.java
28      src/main/java/.../rmi/client/ClienteRMI.java
29  demo/                                        # Mensajeria (WebSocket + STOMP)
30      src/main/java/WebSockets/demo/config/WebSocketConfig.java
31      src/main/java/WebSockets/demo/controller/ChatController.java
32      src/main/resources/static/chat.html

```

9 Conclusión

El taller permitió consolidar, en un solo documento de modelado, los cinco paradigmas de comunicación entre procesos trabajados durante la Unidad 1: Sockets, RPC clásico (Thrift), RPC moderno con streaming (gRPC), RMI y mensajería (WebSocket + STOMP). A diferencia de la primera entrega, aquí cada paradigma quedó formalmente situado dentro de un único escenario de negocio, **UTEQ Connect** (Tabla 1), lo que permitió no solo describir qué hace cada protocolo y middleware, sino justificar comparativamente *por qué ese y no otro* para cada componente del escenario. Esta sistematización deja al equipo en condiciones de argumentar, con criterio técnico y no solo intuitivo, por qué Sockets TCP/UDP es la base más adecuada para iniciar la Práctica Experimental de la Unidad 1, sin descartar el resto de paradigmas para los demás componentes de UTEQ Connect que ya exigen mayor escalabilidad o desacoplamiento.

Referencias

- [1] J. Postel, “Transmission Control Protocol,” RFC 793, Internet Engineering Task Force, Sep. 1981. DOI: 10.17487/RFC0793. [En línea]. Disponible: <https://www.rfc-editor.org/rfc/rfc793>
- [2] J. Postel, “User Datagram Protocol,” RFC 768, Internet Engineering Task Force, Aug. 1980. DOI: 10.17487/RFC0768. [En línea]. Disponible: <https://www.rfc-editor.org/rfc/rfc768>
- [3] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Internet Engineering Task Force, Dec. 2011. DOI: 10.17487/RFC6455. [En línea]. Disponible: <https://www.rfc-editor.org/rfc/rfc6455>

- [4] M. Belshe, R. Peon, and M. Thomson (Ed.), “Hypertext Transfer Protocol Version 2 (HTTP/2),” RFC 7540, Internet Engineering Task Force, May 2015. DOI: 10.17487/RFC7540. [En línea]. Disponible: <https://www.rfc-editor.org/rfc/rfc7540>
- [5] J. Waldo, “Remote procedure calls and Java Remote Method Invocation,” *IEEE Concurrency*, vol. 6, no. 3, pp. 5–7, Jul.–Sep. 1998. DOI: 10.1109/4434.708248.
- [6] M. Slee, A. Agarwal, and M. Kwiatkowski, “Thrift: Scalable Cross-Language Services Implementation,” Facebook, Technical White Paper, Apr. 2007. [En línea]. Disponible: <https://thrift.apache.org/static/files/thrift-20070401.pdf>
- [7] “STOMP Protocol Specification, Version 1.2,” STOMP.github.io, 2012. [En línea]. Disponible: <https://stomp.github.io/stomp-specification-1.2.html>
- [8] Google LLC, “gRPC Documentation,” grpc.io, 2026. [En línea]. Disponible: <https://grpc.io/docs/>
- [9] V. Cerf and R. Kahn, “A Protocol for Packet Network Intercommunication,” *IEEE Transactions on Communications*, vol. 22, no. 5, pp. 637–648, May 1974. DOI: 10.1109/TCOM.1974.1092259.
- [10] A. D. Birrell and B. J. Nelson, “Implementing Remote Procedure Calls,” *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, Feb. 1984. DOI: 10.1145/2080.357392.
- [11] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978. DOI: 10.1145/359545.359563.
- [12] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: Yesterday, Today, and Tomorrow,” in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Cham: Springer, 2017, pp. 195–216. DOI: 10.1007/978-3-319-67425-4_12.

Repositorio de evidencia

Todo el código fuente referido en este documento (Sockets, RPC/Thrift, gRPC, RMI y WebSocket/STOMP) se encuentra disponible públicamente en:

<https://github.com/BedonViteri/Aplicaciones-Distribuidas>